

P.O.O / Java — Programação Orientada a Objetos

Aula 13 — Polimorfismo

Nome: _____ Data: _____

1. Polimorfismo

Polimorfismo (do grego: muitas formas) permite que objetos de classes diferentes sejam tratados através de um tipo comum, executando cada um seu próprio comportamento.

Java

```
// Usando a hierarquia da aula anterior
public class Main {
    public static void main(String[] args) {

        // Polimorfismo: referência do tipo pai aponta para objeto filho
        Pessoa p1 = new Aluno("Maria", 17, 8.5, "2A");
        Pessoa p2 = new Professor("João", 35, "Informática");

        // Cada um chama sua própria versão de apresentar()
        p1.apresentar(); // chama apresentar() de Aluno
        p2.apresentar(); // chama apresentar() de Professor

        // Array polimórfico
        Pessoa[] pessoas = {
            new Aluno("Ana", 16, 9.0, "1B"),
            new Professor("Carlos", 40, "Matemática"),
            new Aluno("Pedro", 17, 7.5, "2A")
        };

        // Loop polimórfico – cada objeto responde ao seu modo
        for (Pessoa p : pessoas) {
            p.apresentar();
            System.out.println("---");
        }
    }
}
```

2. instanceof: verificando o tipo em tempo de execução

Java

```
for (Pessoa p : pessoas) {  
    if (p instanceof Aluno) {  
        Aluno a = (Aluno) p; // casting para acessar métodos específicos  
        System.out.println("Aluno com nota: " + a.getNota());  
    } else if (p instanceof Professor) {  
        System.out.println("Professor");  
    }  
}
```

Exercício 1 — Polimorfismo na prática

Usando a hierarquia de Veiculo, crie um array de Veiculo[] com 2 carros e 2 motos. Percorra o array chamando `exibir()` para cada um. Use `instanceof` para contar quantos são Carro e quantos são Moto:

Resumo da aula

- Polimorfismo: referência do tipo pai pode apontar para qualquer subclasse.
- O método chamado é determinado em tempo de execução (dynamic dispatch).
- `instanceof` verifica o tipo real do objeto em tempo de execução.